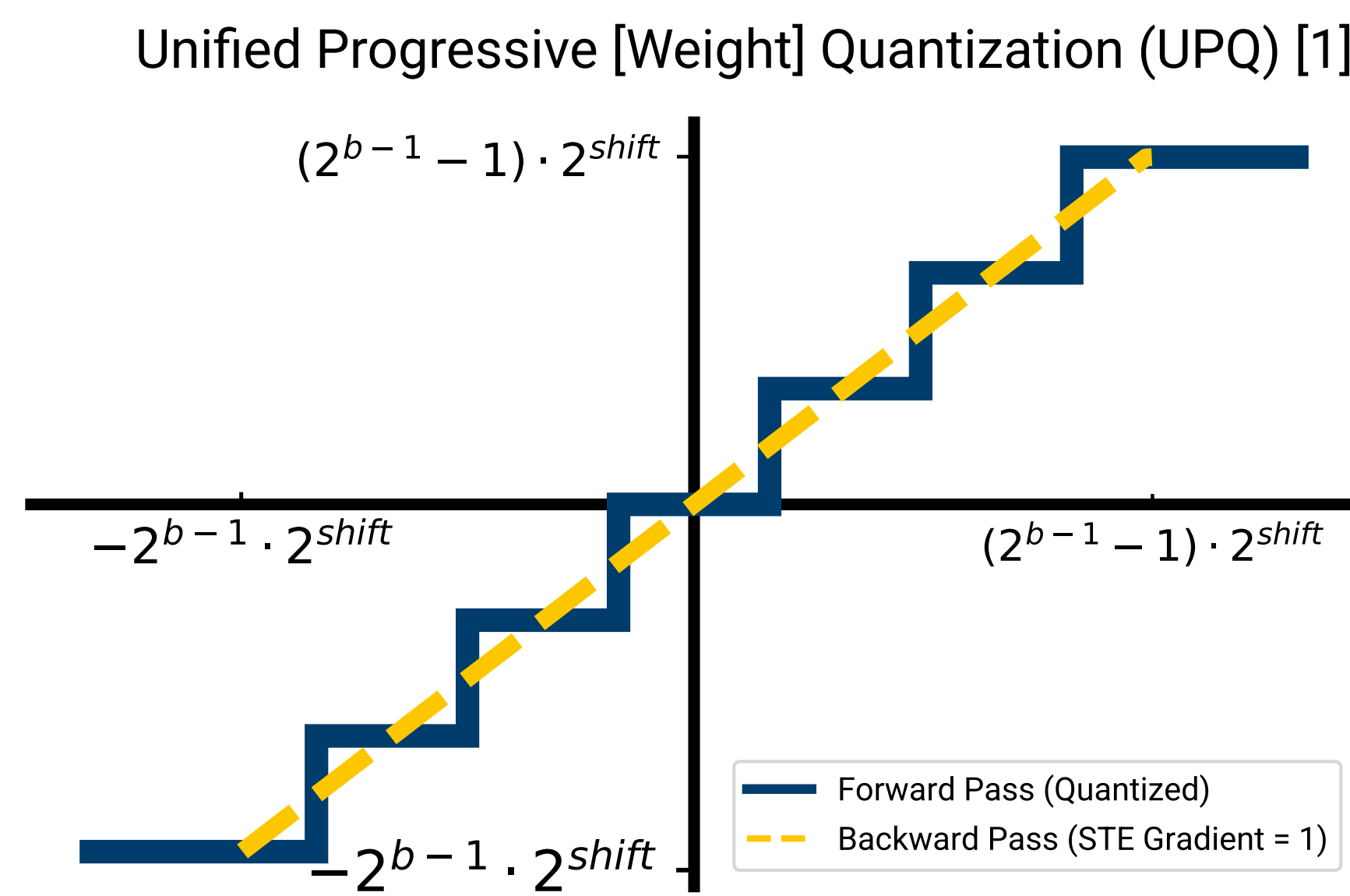


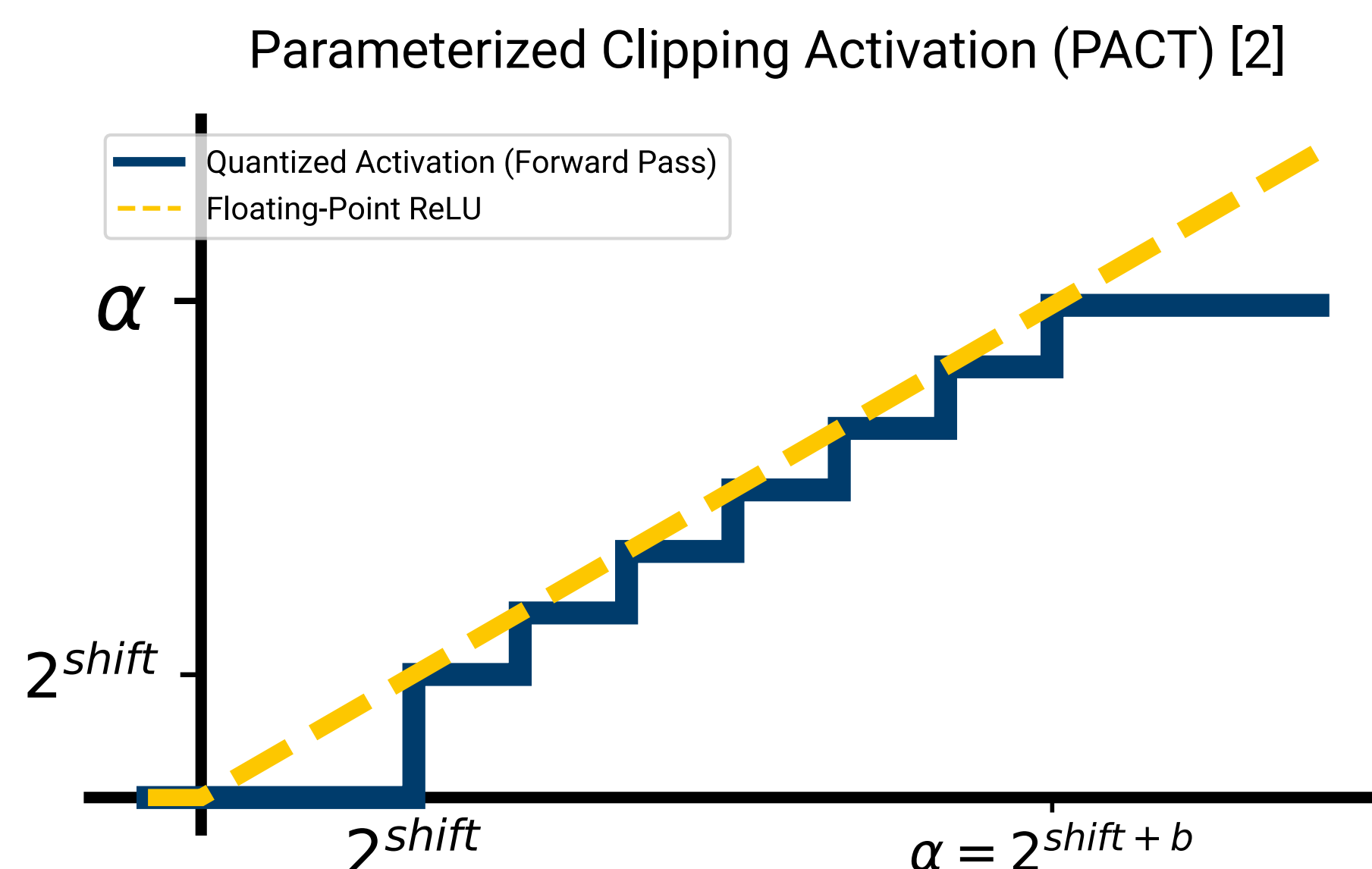
Introduction

This project introduces a novel software-hardware co-design environment that interfaces an FPGA with a microcontroller to enable real-time, edge-based computer vision without relying on cloud infrastructure. Within this environment, Convolutional Neural Networks (CNNs) are trained in PyTorch with quantized parameters, and seamlessly transformed into an equivalent Verilog implementation. This parameterized streaming architecture fully utilizes the FPGA's limited compute resources (LCs, DSPs, BRAMs) to perform inference on a binarized live input feed from the microcontroller. Finally, design time is drastically reduced by profiling each module's hardware overhead, generating all valid network topologies, and ranking them based on highly accurate performance predictions.

Network Quantization

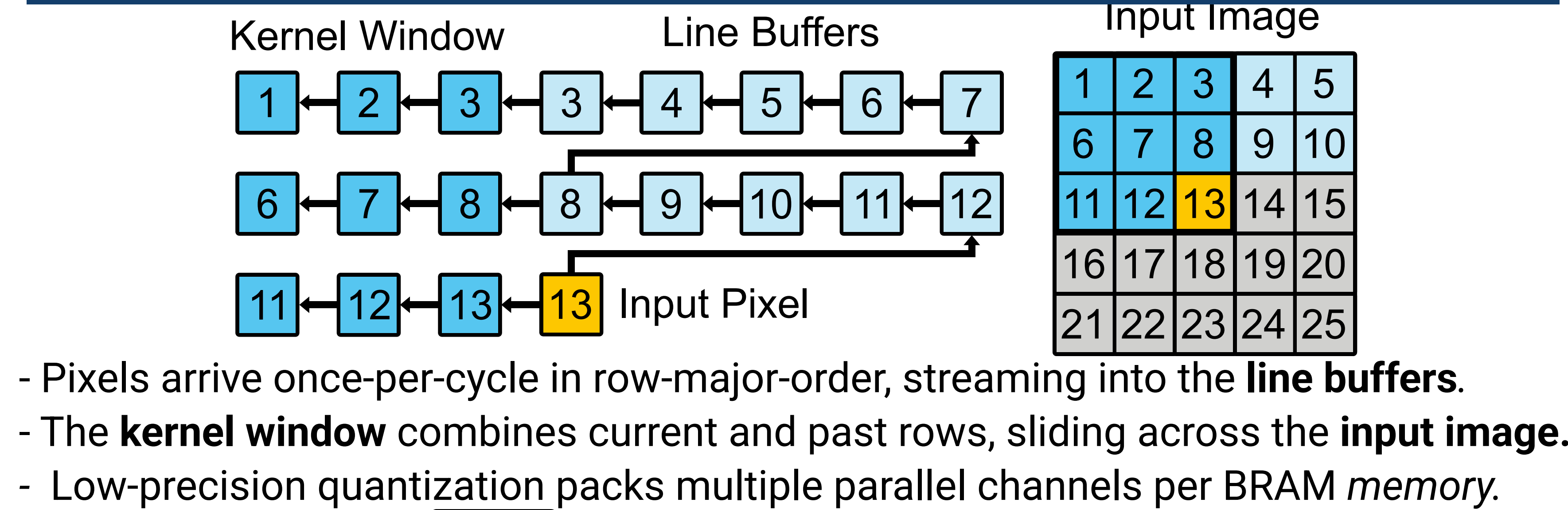


- Weights start as floating points, then progressive QAT gradually reduces precision as low as ternary (2-bit).
- Learned weights captured as integers and exported to verilog ROM files for simulation and synthesis.

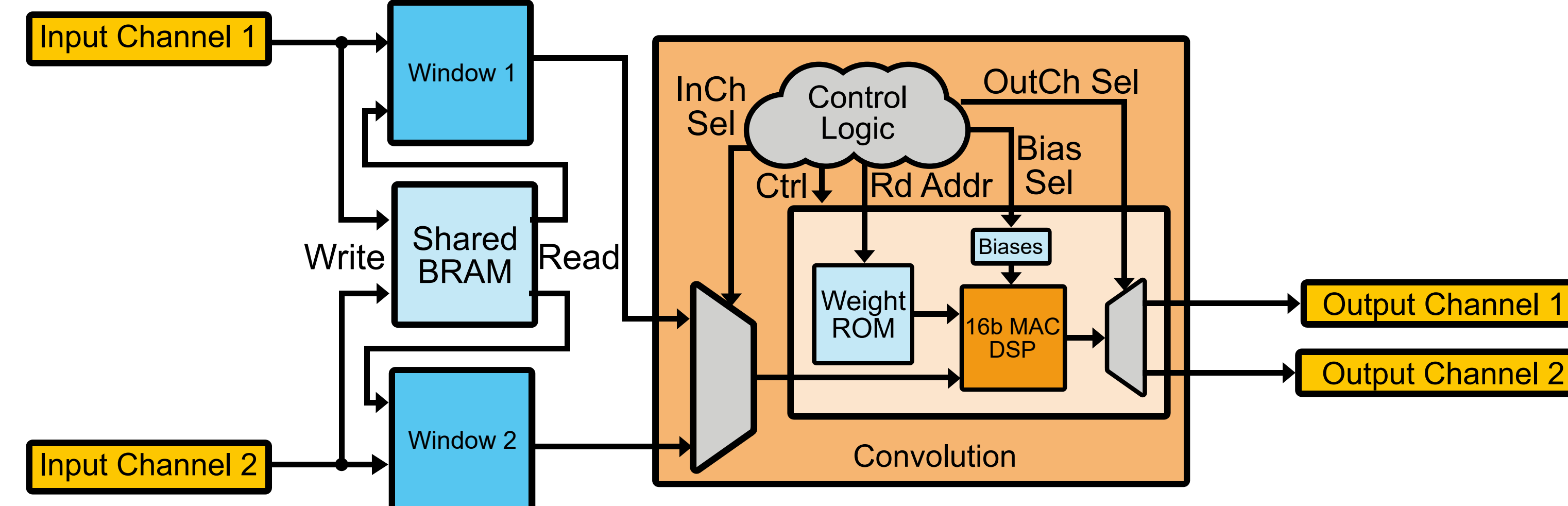


- During training, each neural network layer dynamically learns the optimal clipping range (2^b) and scale ($1/(2^{\text{shift}})$).
- In hardware, the scale and shifted range are exported as Verilog Parameters for each layer.

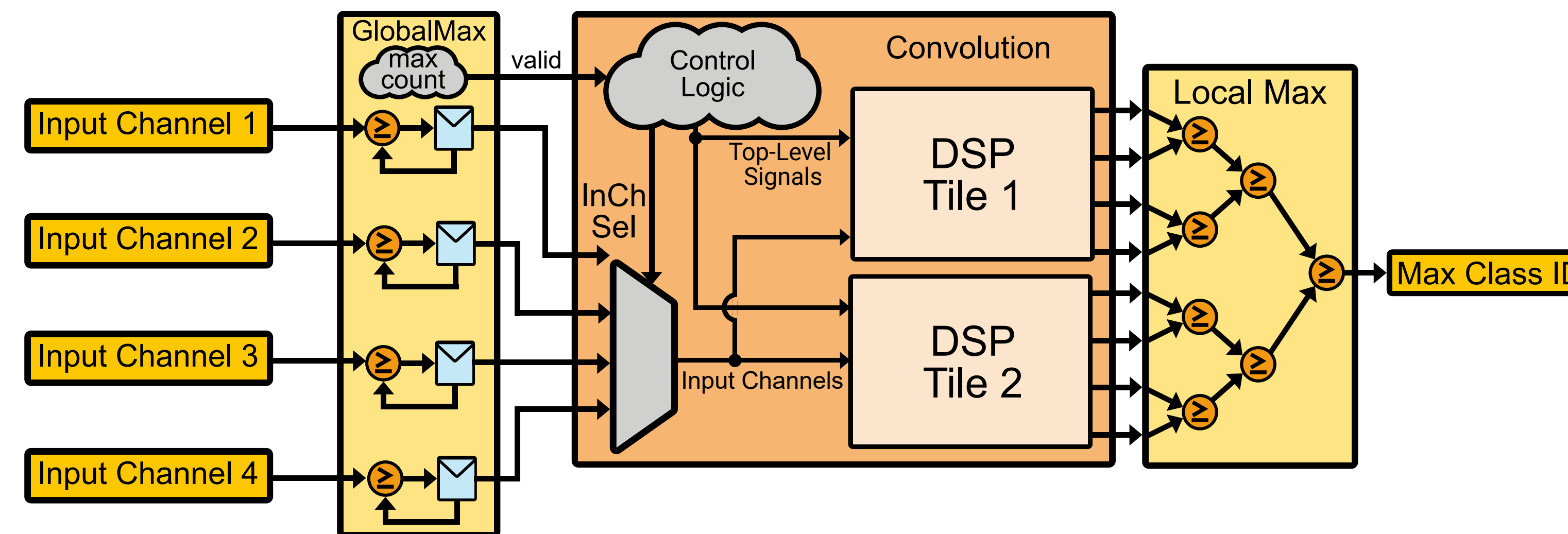
System Architecture



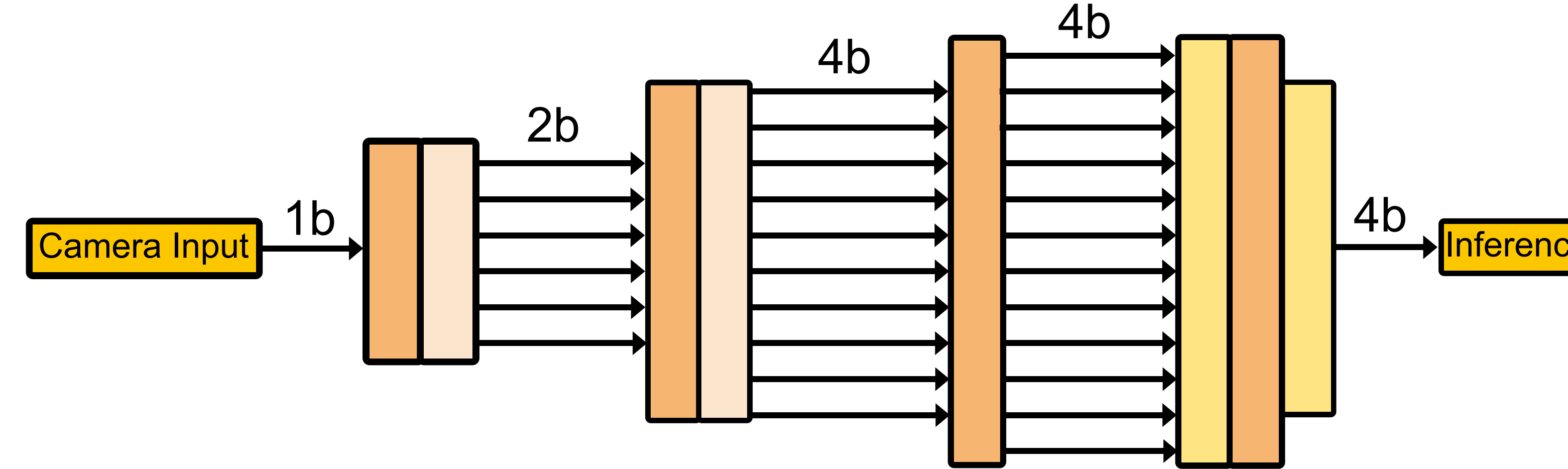
- Pixels arrive once-per-cycle in row-major-order, streaming into the **line buffers**.
- The **kernel window** combines current and past rows, sliding across the **input image**.
- Low-precision quantization packs multiple parallel channels per BRAM *memory*.



- **Convolution layers** compute dot products between the sliding window and weights.
- Combinational mode offers single-cycle latency, optimal for lower input bandwidth.
- Sequential mode trades DSP blocks and added latency to save Logic Cells (LC).
- **Pooling layer** data movement is identical, but performs a max instead of dot product.



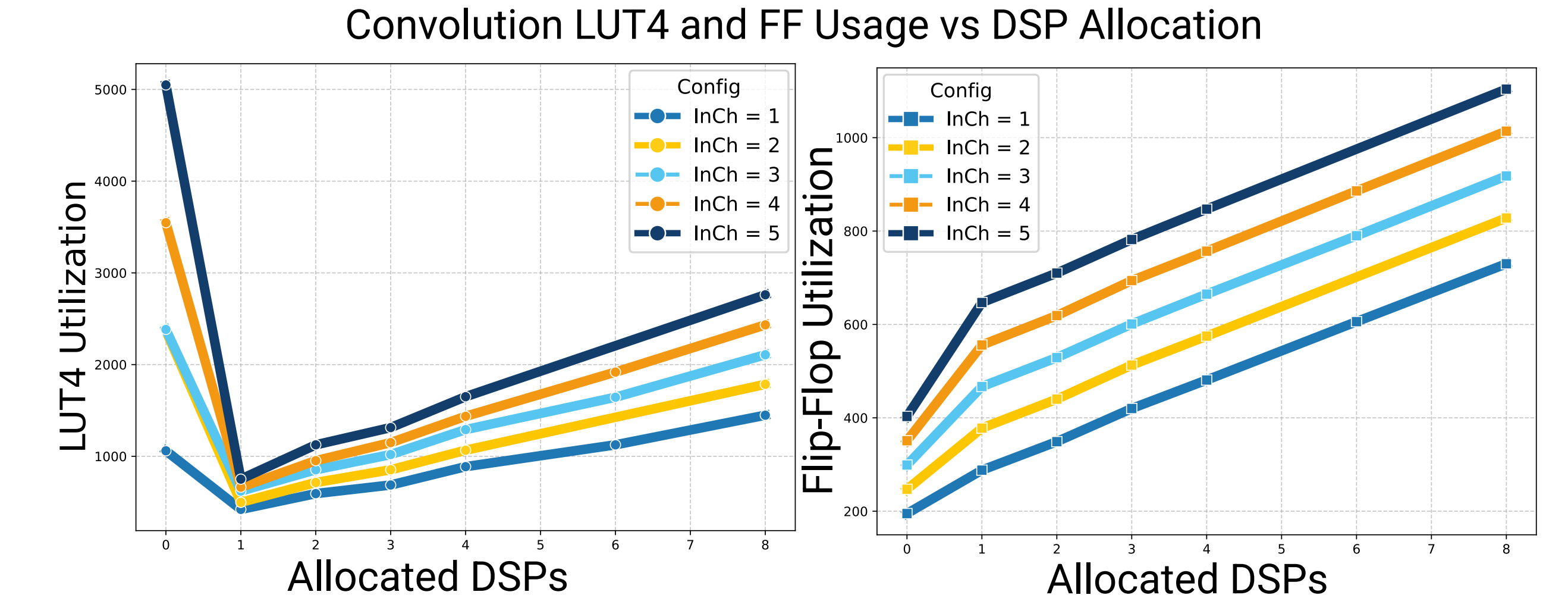
- **Classifier layer** connects sequential **global max** with 1x1 convolution and **local max**.
- Latency can be reduced by dividing the workload over multiple DSPs.



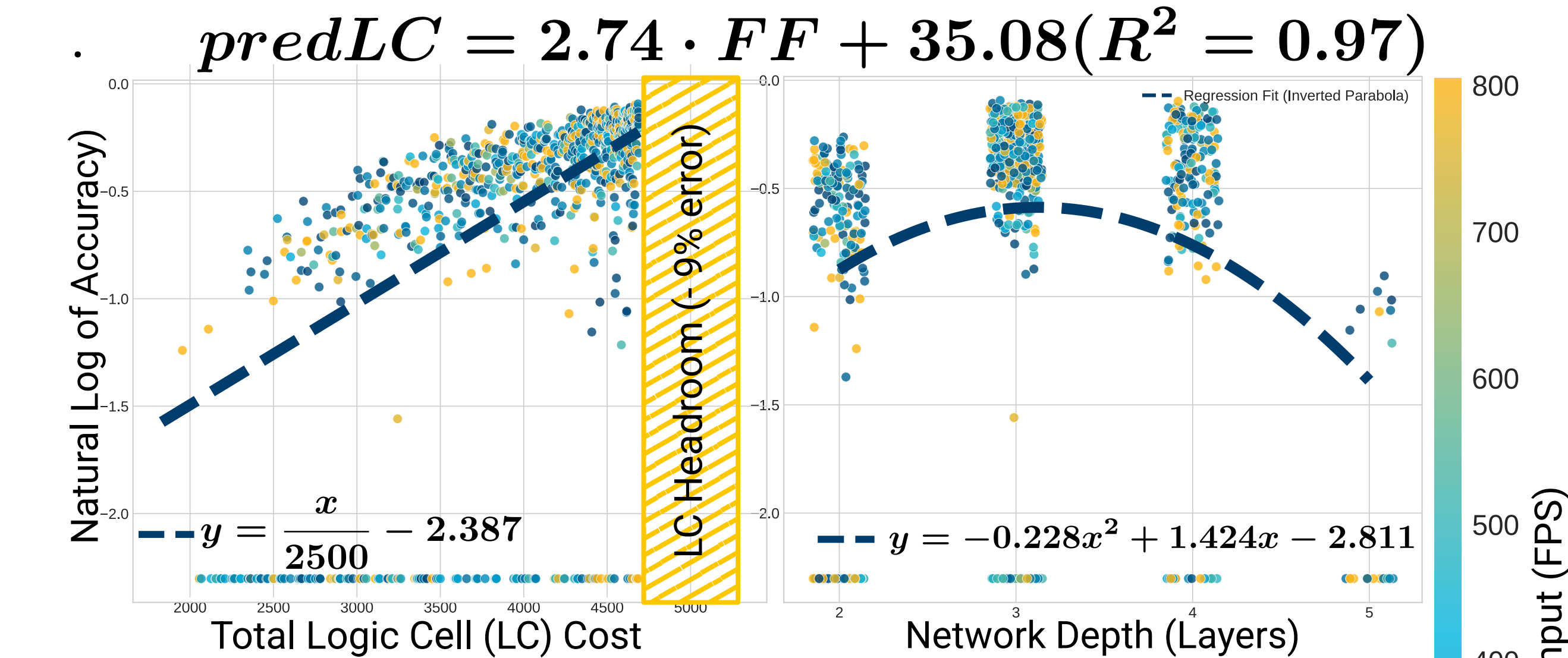
- PyTorch network mapped to Verilog, confirming it fits within FPGA hardware budget.

Layer	C0	P0	C1	P1	C2	Classifier	Total
Logic Cells	301	301	666	672	1114	822	4330/4806
Block RAMs	1	1	5	3	10	1	21/29
DSPs	0	0	1	0	1	1	3/8

Design-Space Exploration



- Synthesized each module over all parameter combinations.
- Trained regression models to predict FF and LUT4 costs.
- Developed a constrained DFS to generate all real networks.



- Trained resulting networks on the MNIST dataset.
- Regression model then trained to predict accuracy from network architecture.
- Designers can select from top networks for throughput.
- Architectures that fully utilize logic cells perform better.
- Ternary activations allow for deeper networks, but gradients vanish, with accuracy peaking at networks with three layers.

$$\ln(\text{acc}) = \frac{\text{LC}}{2500} - \frac{\text{Ternary}\%}{117} - 0.226 \cdot \text{depth}^2 + 1.422 \cdot \text{depth} - 4.24 (R^2 = 0.53)$$

Results

By integrating this network sweep and accuracy prediction within an automated toolchain, designers can rapidly deploy the globally optimal architecture for accuracy and throughput. Benchmarking using the MNIST dataset yields real quantized accuracy of 95% and 72 fps. This exact accuracy was rigorously verified through cycle-accurate simulations, validated physically on the FPGA, and fully integrated for low-power edge inference via the ESP32c3 and Arducam frontend.